RasoAI: An Ingredient-Based Indian Recipe Retrieval System

Shardul Kamde, Samvat Jain, Abhishek Ghatti, and Aryan Singh
Department of Computer Science and Engineering
Pimpri Chinchwad University
Pune, India

***Abstract*—RasoAI is a local recipe retrieval application that helps users find an Indian recipe using ingredients they have available. It searches a workbook containing 6,871 recipe records and returns one eligible match with its source metadata and original instructions. A Streamlit interface collects searchable dataset ingredients or custom entries and sends them to a FastAPI service. The backend normalizes ingredient text, uses RapidFuzz string similarity to accommodate limited spelling variation, filters candidates by a minimum ingredient-coverage threshold, and ranks eligible recipes by coverage and keyword overlap. The system retrieves recipe content from the workbook rather than generating instructions with a language model. Four unit tests exercise exact matching, query-oriented ranking, minimum coverage, and spelling variation. The current implementation is a practical local search prototype; its lexical score is heuristic and has not been evaluated as a measure of recipe quality or user satisfaction.**

***Index Terms*—recipe retrieval, ingredient matching, content-based recommendation, fuzzy string matching, FastAPI, Streamlit**

I. Introduction

Recipe search commonly begins with a dish name, whereas a home cook may begin with ingredients already on hand. RasoAI addresses this ingredient-first use case by searching a structured Indian-food workbook and presenting a recipe that overlaps with the user's input. It is designed as a local application and does not depend on a hosted generative model at runtime.

The project combines a searchable ingredient-selection interface, an HTTP API, and a deterministic retrieval component. Its contribution is an end-to-end implementation that connects these pieces while preserving the recipe instructions and metadata from the source records. It is not intended to establish a new recommendation algorithm or to claim that text similarity guarantees culinary suitability.

II. Related Work

Content-based recommendation ranks items according to features associated with the item and the user's stated interests [1]. Ingredient-based recipe retrieval applies this general idea by comparing a query with recipe ingredients. Text normalization and token matching are common information-retrieval techniques for reducing variation in textual queries [2]. RasoAI uses these ideas in a deliberately lightweight form: normalized ingredient keywords and a fuzzy string threshold, rather than a trained ranking model.

The implementation uses RapidFuzz for approximate string comparison [3]. Pandas and OpenPyXL load the workbook [4], [5]; FastAPI exposes the backend endpoints [6]; and Streamlit provides the user interface [7].

III. System Overview

The application has three runtime components. First, the Streamlit frontend lets a user search or select ingredients from suggestions derived from the dataset, and also accepts comma-separated custom ingredients. Second, the FastAPI backend loads the Excel workbook and exposes the available ingredient list and recipe-search operations. Third, the retrieval component normalizes the query and recipe ingredients, finds eligible candidates, and returns the highest-ranked record. The frontend displays recipe metadata, the source URL when available, the dataset ingredient text, and the original instructions as readable steps.

TABLE I
RasoAI API operations

Operation	Purpose
`GET /ingredients`	Returns dataset-derived ingredient choices for the interface.
`POST /chat`	Accepts an ingredient list and returns a best match or a no-match response.

IV. Dataset and Preprocessing

The application reads `IndianFoodDatasetXLS.xlsx`, which contains 6,871 rows. The loader requires recipe identifiers, names, ingredients, instructions, cuisine, course, diet, preparation and cooking times, servings, and source URLs. It constructs an in-memory recipe record for each row when the backend starts.

Ingredient text is lowercased; parenthetical content, digits, punctuation, and repeated whitespace are removed. A stopword list filters measurement and preparation terms, and tokens of two characters or fewer are discarded. The remaining tokens are used for matching. The source ingredient and instruction text is retained for display, so normalization affects retrieval but does not rewrite the recipe shown to the user.

V. Matching Method

Let Q be the submitted ingredient list after entries with no extractable keywords are removed. For each recipe r, the

retriever compares the keywords extracted from each query ingredient with the recipe's keyword set. A query ingredient counts as matched only when all of its extracted keywords have a recipe keyword with a RapidFuzz ratio of at least 90.

Candidate coverage is

$$C(Q, r) = 100 \frac{m(Q, r)}{|Q|}, \tag{1}$$

where $m(Q, r)$ is the number of matched query ingredients. Candidates with coverage below 50% are discarded. For each eligible candidate, the retriever also calculates a Dice-style keyword-overlap score:

$$S(Q, r) = 100 \frac{2\, k(Q, r)}{|K_Q| + |K_r|}, \tag{2}$$

where K_Q and K_r are the sets of query and recipe keywords, and $k(Q, r)$ counts query keywords that match a recipe keyword at the same fuzzy threshold. Candidates are ranked lexicographically by (C, S): ingredient coverage is the primary criterion and the keyword score breaks ties. The displayed score is this lexical score; it is not a probability or a percentage of all recipe ingredients present.

If no candidate satisfies the coverage threshold, the API returns a no-match response. Otherwise, it returns one best-match record, including recipe metadata, dataset ingredients, instructions, and the source URL.

VI. Implementation

The backend is implemented in Python with FastAPI and Pydantic. It loads the workbook with pandas and relies on OpenPyXL for Excel parsing. The retriever uses RapidFuzz for pairwise token comparisons. The Streamlit frontend calls the backend over HTTP, provides both dataset-backed suggestions and custom ingredient entry, and presents recipe details and numbered instructions. The system therefore separates the user interface from the retrieval logic while remaining deployable as two local processes.

The matching behavior is deterministic for a fixed workbook and query. No recipe content is synthesized: instructions and ingredient text returned to the user come from the selected workbook record. The application does not currently use the dataset's translated fields for retrieval.

VII. Functional Validation and Results

The workbook contains 6,871 rows and includes the fields used for retrieval and result display. The project includes four focused unit tests for the retriever. Table II summarizes the behaviors they check.

These tests validate selected functional rules, not recommendation quality across a representative set of real user queries. No precision, recall, latency, or user-satisfaction benchmark is claimed. The score and thresholds are configuration choices that require broader evaluation before they can be interpreted as measures of relevance.

TABLE II
Recipe-retriever unit-test coverage

Test	Expected behavior
Exact ingredient preference	Prefer a recipe containing the exact requested ingredient over a partial-word candidate.
Query-based ranking	Prefer the recipe that better covers the supplied ingredients.
Minimum coverage	Return no match when fewer than half of the submitted ingredients match.
Spelling variation	Accept a small spelling variation that passes the configured fuzzy threshold.

VIII. Limitations and Future Work

The system depends on a static workbook loaded into memory at backend startup; workbook changes require a restart. Matching is lexical and can miss synonyms, regional ingredient names, transliterations, and meaningful semantic relationships. The 50% coverage rule may admit an unsuitable recipe or reject a useful one, and the retriever returns only one result. In addition, the application does not validate allergies, dietary restrictions, nutrition, or food safety. Dataset labels such as diet are displayed as supplied and are not independently verified.

Future work includes evaluating ranking against manually judged ingredient queries, returning several ranked candidates, improving ingredient parsing and synonym handling, and adding integration tests for the API and interface. Dietary and allergy filtering would require a carefully validated data source and should not be inferred from the current lexical matching score.

IX. Conclusion

RasoAI demonstrates a local, ingredient-first recipe retrieval workflow built around a structured Indian-food workbook. A Streamlit interface sends user-selected or custom ingredients to a FastAPI backend, where normalized text and fuzzy keyword comparisons identify candidates meeting a minimum coverage threshold. The system ranks eligible records by coverage and keyword overlap and presents the original dataset content. Its tests cover core matching behaviors, while broader relevance and safety evaluation remain open work.

References

[1] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015, doi: 10.1007/978-1-4899-7637-6.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/

[3] RapidFuzz, "RapidFuzz documentation." [Online]. Available: https://rapidfuzz.github.io/RapidFuzz/

[4] The pandas development team, "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs/

[5] openpyxl, "openpyxl documentation." [Online]. Available: https://openpyxl.readthedocs.io/

[6] FastAPI, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[7] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/